

POLITECNICO
MILANO 1863

Best Bike Paths – DD Document

DD document for the
Software Engineering 2 course assignment

Authors: **Alessandro Burato, Gabriele Crimella, Davide Faccini**

Teacher: Elisabetta Di Nitto
Academic Year: 2025-26

Contents

Contents **i**

1 Introduction **1**
1.1 Purpose . . . 1
1.2 Scope . . . 1
1.3 Definitions, Acronyms, Abbreviations . . . 1
1.4 Reference Documents . . . 2
1.5 Document Structure . . . 2

2 Architectural Design **3**
2.1 Overview . . . 3
2.2 Component View . . . 3
2.2.1 Mobile . . . 3
2.2.2 Backend . . . 3
2.3 Deployment View . . . 3
2.4 Runtime View . . . 4
2.5 Component Interfaces . . . 4
2.6 Styles and Patterns . . . 5
2.7 Other Design Decisions . . . 6
2.7.1 JWT . . . 6
2.7.2 Avoidance of Single Points of Failure . . . 6
2.7.3 Microservices . . . 6
2.7.4 Cross-Platform Availability . . . 6

3 User Interface Design **7**

4 Requirements Traceability **9**

5 Implementation, Integration and Test Plan **11**
5.1 Overview and Implementation Plan . . . 11
5.2 Features identification . . . 11

5.3 Integration Strategy 11
5.4 System Testing Strategy 11

6 Effort Spent **13**

7 References **15**

List of Figures **17**

List of Tables **19**

1 Introduction

1.1. Purpose

The purpose of this document is to serve as a detailed descriptor of the BBP project, to be used by actual developers and as an actual agreement between customer and contractors: it will contain detailed informations about the architecture of the system, its full functionalities, constraints and it will allow the customer to validate the integrity and completeness of said system. With this DD, the customer will also be able to validate all of the requirements, to verify if the system finally meets the expectations.

1.2. Scope

The Best Bike Paths (BBP) project, initiated by a leading bike association, aims to be an advanced support system for the cycling community designed to thrive within a transport landscape dominated by cars.

The core idea is to foster a collaborative digital environment where cyclists can actively record, consult, and publish both their own and other users' riding paths.

This platform will serve as the definitive centralized resource for finding highly recommended routes tailored precisely to the user's specific experience, preferences, and safety considerations.

Ultimately, the system aims to significantly enhance the quality of the cycling experience by providing verified, community-rated routes, thereby connecting riders and making high-quality path discovery universally accessible

1.3. Definitions, Acronyms, Abbreviations

- BBP: Best Bike Paths
- RASD: Requirements Analysis and Specification Document
- DD: Design Document
- IIT: Implementation, Integration and Test Plan

- WAF: Web Application Firewall

1.4. Reference Documents

- Specification document assignment
- IEEE standard documentation for DD
- Materials for the Software Engineering 2 course made available on WeBeep

1.5. Document Structure

The document is organized in seven different chapters that will cover every detail on the actual design of our project:

- **Chapter 1 - Introduction:** this chapter shows the aim of the doucment, what it is needed to know before to actually start the read of the whole text and restates the scope of the project.
- **Chapter 2 - Architectural Design:** this chapter shows all of the main components of the whole system, including architectural choices and patterns used.
- **Chapter 3 - User Interface Design:** this chapter shows and explains how the mockup pages we created integrate all of the implemented functionalities.
- **Chapter 4 - Requirements Traceability:** this chapter shows how we ensured that every requirement needed to the system to work is being satisfied with relative design choices.
- **Chapter 5 - Implementation, Integration and Test Plan:** this chapter shows how all of the components are implemented and their integration inside the system, also providing a test plan for all of their instances.
- **Chapter 6 - Effort Spent:** this chapter includes a table where the amount of hour of commitment of every participant is shown relative to its section.
- **Chapter 7 - References:** finally, the last chapter contains all of our document references used to better refine our DD.

2 Architectural Design

2.1. Overview

2.2. Component View

2.2.1. Mobile

2.2.2. Backend

2.3. Deployment View

In this section of the document we focus on the deployment architecture of BBP and the motivations behind our design choices.

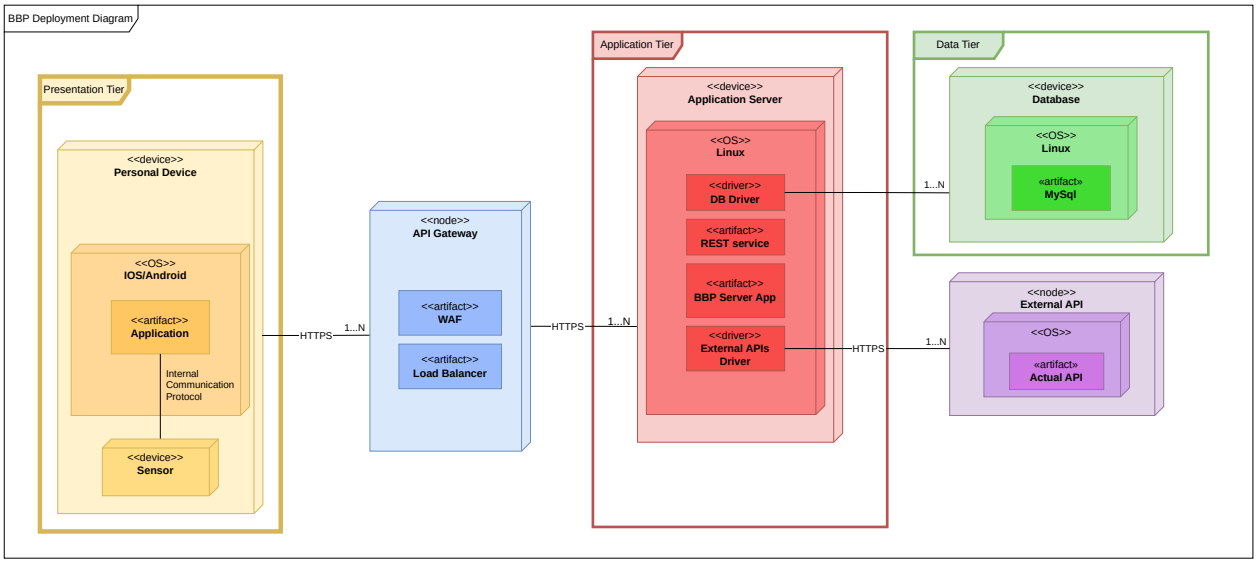


Figure 2.1: Deployment View

We divided our system into three main tiers—presentation, application, and data tiers (better explained in Section 2.6)—to better isolate common features between all tangible and intangible instances of the objects involved. Below is a list of all actors and components in the system:

- **Personal Device**: a personal device consists of a sensor device (here modeled as the union

of all the device's movement sensors) and an operating system that can be either Android or iOS. It is intended as a personal piece of hardware such as a smartphone or smartwatch carried by the user throughout the execution of the application. The personal device communicates with the Application Server through a local instance of the application: every request made by the Application to the Application Server follows a RESTful interaction style over HTTPS.

- **API Gateway**: it is a node offered by a third party (such as AWS or Google Cloud Services) that provides a built-in Web Application Firewall (WAF) and a load balancer to distribute traffic among multiple Application Servers, thus avoiding single points of failure.
- **Application Server**: it is a device that contains the core logic of the application and connects all the drivers required for the application to operate. It communicates with external APIs using REST and interacts with multiple databases through a continuously managed connection pool.
- **Database**: we opted for multiple databases (one primary database and several read-only replicas) with a connection pool managed by the Application Server, ensuring that databases are protected by a limited number of open connections. This approach minimizes concurrency issues and prevents overload. The number of connections can be adjusted according to traffic patterns to achieve an optimal balance between hardware usage and system efficiency. Linux was chosen as the operating system due to its open-source nature, and MySQL for its robustness.

A closer look at some of the internal components of the system follows:

- **Application Artifact**: the client-side application, primarily used as the presentation layer. It stores a signed JWT (JSON Web Token) used for authentication and is connected to all sensors required to detect movements in automated mode using cross-platform mobile frameworks.
- **BBP Server App**: the core model of the application. It contains the business logic and ensures that all operations are coherent with user requests by following the Model-View-Controller (MVC) pattern. It hosts all managers and connects them to the necessary drivers used to interface with external tools.

2.4. Runtime View

2.5. Component Interfaces

2.6. Styles and Patterns

BBP will be developed using the client-server paradigm integrated with a three-tier architecture and the Model-View-Controller pattern. The goal of this combined architecture is to achieve a clear separation of concerns among system components and improve control over each tier. The tiers involved are:

- **Presentation Tier**: this tier serves as the view component of the application and contains software that interacts directly with the user's hardware (i.e., the personal device and its sensors). It represents the top-level tier and is designed to be mostly stateless, although some user information is stored through JWT tokens for authentication. Its main functionality is to display information and available actions to the user, as well as present outputs generated by the backend. The presentation tier does not contain core business logic, preventing users from modifying it locally to perform unauthorized actions and keeping the application lightweight in terms of memory usage. To ensure platform independence, Flutter was adopted to provide a unified abstraction layer for accessing sensor information across operating systems.
- **Application Tier**: this tier contains the core logic of the system and is composed of the Application Servers. It validates and executes user actions according to the current system state. Acting as a middle tier, it processes data received from the presentation tier and returns meaningful information to the user. Multiple Application Servers are deployed to enable scalability; with the use of an API Gateway equipped with a load balancer, requests can be routed consistently to specific servers using the JWT as an identifier. In case of server failure, users can be transparently rerouted to other servers, avoiding downtime.
- **Data Tier**: this tier represents the main repository of user and application data. It consists of a primary database and multiple secondary read-only replicas, all accessed through a connection pool that regulates the number of active connections. This architecture avoids data-tier downtime by allowing replica promotion in case of primary database failure, while preventing data inconsistency issues that may arise from having multiple writable databases.

In addition to the three tiers, a third-party API Gateway connects the presentation tier to the application tier. To satisfy scalability and security requirements, the API Gateway includes a **load balancer** to distribute incoming connections and a **Web Application Firewall** to protect against common attacks. AWS and Google Cloud Services are examples of providers offering such solutions.

The overall architecture follows a **client-server** interaction model in which most operations are event-driven. The client initiates the majority of transactions, except for server-generated

notifications (e.g., automated mode activation). All client requests are validated by server-side logic, ensuring that communication errors and malicious behaviors are intercepted. Since communication is handled through **REST** over HTTPS, error messages can be clearly encoded and traced in both front-end and back-end scenarios.

The primary design pattern adopted is **Model-View-Controller**. The Model contains the logical components that manage and process application data, the View represents the graphical interface presented to the user through the client application, and the Controller handles user input and coordinates interactions between the Model and the View.

2.7. Other Design Decisions

2.7.1. JWT

JWT was chosen for authentication because, when combined with the load balancer deployed within the API Gateway, it enables secure login and registration while supporting scalability. Token expiration times (time-to-live) can be customized to allow either long-lived sessions or frequent re-authentication, providing flexibility in security policies.

2.7.2. Avoidance of Single Points of Failure

The use of an API Gateway, a homogeneous set of Application Servers, and load-balanced routing eliminates single points of failure. This design allows the system to scale horizontally by adding servers, thereby increasing robustness and the ability to handle higher request volumes.

2.7.3. Microservices

A microservices-based architecture was adopted to improve functional scalability. Each manager is implemented as an isolated service coordinated through notification and query mechanisms. This approach facilitates targeted testing, simplifies maintenance, and improves error identification and resolution.

2.7.4. Cross-Platform Availability

By using Flutter, a single codebase is maintained for the client-side application, improving code readability and maintainability through a shared operating system abstraction. This choice also enables BBP to reach a wider user base by supporting multiple platforms.

3 | User Interface Design

4 | Requirements Traceability

5 Implementation, Integration and Test Plan

5.1. Overview and Implementation Plan

5.2. Features identification

5.3. Integration Strategy

5.4. System Testing Strategy

6 Effort Spent

A table representing the total hours of work is shown here.

Work Areas	Alessandro Burato	Gabriele Crimella	Davide Faccini
Introduction	1h	0h	0h
Architectural Design	8h	0h	0h
UI Design	0h	0h	0h
Requirements Traceability	0h	0h	0h
IIT	0h	0h	0h
Group Meetings and Reviews	0h	0h	0h

Table 6.1: Table of Effort

7 | References

List of Figures

2.1 Deployment View 3

List of Tables

6.1 Table of Effort 13